

Instituto Federal de Educação, Ciência e Tecnologia
Curso: Bacharelado em Ciência da Computação
Disciplina: Projeto e Análise de Algoritmos
Professor: Mário Luiz Rodrigues Oliveira
Atividade: Lista Exercícios I
Formiga,MG, 11 de agosto de 2026

Instruções:

1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.

2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.

3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1. Responda **V** para os itens que são verdadeiros e **F** para os itens que são falsos e justifique suas respostas. Respostas sem justificativas não serão consideradas.

(a) ( ) Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em algumas entradas?
(b) ( ) Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em todas as entradas?
(c) ( ) Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em algumas entradas?
(d) ( ) Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em todas as entradas?
(e) ( ) Se eu provar que um algoritmo leva $\theta(n^2)$ no pior caso, é possível existir alguma entrada suficientemente grande para a qual o algoritmo realiza pelo menos $100 * n$ operações primitivas?
(f) ( ) Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, há obrigatoriamente alguma entrada suficientemente grande para a qual o algoritmo realiza pelo menos $100 * n$ operações primitivas?

Questão 2. Considere n representando um valor variável e k é um valor constante. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação *Big O*. Dê um limite assintótico tão firme quanto possível. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.

- **código 1:**

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
{
    for (j = 1; j <= n; j = j + 1)
    {
       soma = soma + 1;
    }
}
```

- **código 2:**

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
{
    for (j = 1; j <= n; j = j + 1)
    {
        for (c = j; c <= k; c = c + 1)
        {
            soma = soma + 1;
        }
    }
}
```

- **código 3:**

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
{
    for (j = 1; j <= i; j = j + 1)
    {
       soma = soma + 1;
    }
}
```

- **código 4:**

```
soma = 0;
for (i= 1; i <= n; i = i + 1)
{
    for (j = i; j <= n; j = j + 1)
    {
        c:= 1;
        while ( c <= k )
        {
            soma = soma + 1;
            c = c * 2;
        }
    }
}
```

- **código 5:**

```
soma = 0;
for ( i= 1; i <= n; i = i + 1)
{
    for (j = 1; j <= n; j = j + 1)
    {
        c = 1;
        while ( c <= n )
        {
            soma = soma + 1;
            c = c * 5;
        }
    }
}
```

Questão 3. Demonstre as seguintes propriedades relativas à notação assintótica.

(a) $f(n) = 2^{117},\ f(n) \in O(1)$
(b) $f(n) = n^2 + 2 * n,\ f(n) \in O(n^2)$
(c) $f(n) = n^2 - 2 * n + 3,\ f(n) \in \theta(n^2)$
(d) $f(n) = 5 * n,\ f(n) \in \Omega(n)$
(e) $f(n) = n^2 + 2 * n - 8,\ f(n) \in \Omega(n^2)$

Questão 4. Suponha que um algoritmo para um certo problema consome $O(n^2)$ unidades de tempo, sendo n o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome apenas $\Omega(n)$ unidades de tempo. Devo ficar impressionado(a)?

Questão 5. Meu algoritmo para um certo problema consome $O(n^2)$ unidades de tempo, sendo n o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome $O(n * lgn)$ unidades de tempo. Devo ficar impressionado(a)?

Informações que podem ser úteis

Análise de Algoritmos:

Não há um conjunto de regras para analisar um algoritmo. No entanto, conforme [AHO et al., 1983] podemos seguir o seguinte guia, considerando o modelo computacional *Random Access Machine* (RAM):

1. O custo de execução de uma instrução de atribuição, de leitura ou de escrita é O(1);
2. O custo de execução de uma sequência de instruções é determinado pela instrução de maior custo na sequência de instruções;

3. O custo de execução de um comando de decisão/seleção é composto pelo custo de execução das instruções no corpo do comando de decisão/seleção mais o custo para avaliar a condição. O custo para avaliar a condição é O(1), e
4. O custo de execução de um comando de repetição (laço/loop) é a soma do custo de execução do corpo do comando de repetição adicionado ao custo de avaliar a condição de término do comando de execução, multiplicado pelo número de iterações do comando de repetição. O custo para avaliar a condição de término é O(1).

Somatórios e Logaritmos:

- $\sum_{i=1}^{n}(i) = n(n+1)/2$
- $\sum_{i=1}^{n}(a^i) = a(a^n - 1)/(a-1)$

- $\log(a * b) = \log a + \log b$
- $\log(a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:
$1 \prec log(n) \prec \sqrt{n} \prec n \prec n * log(n) \prec n^c \prec c^n \prec n! \prec n^n$ onde c é uma constante arbitrária inteira ≥ 2.

Referências

[AHO et al., 1983] AHO, A. V., HOPCORFT, J. E., and ULLMAN, J. D. (1983). *Data Structures and Algorithms.* Addison-Wesley.